Day 168: Grid Search against Random Search: a Real Efficiency Comparison

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 1 | Complete study guide

Watch: Hyperparameters Tuning: Grid Search vs Random Search (DataMListic) | IML17: Grid vs random search for hyperparameter tuning? (Bevan Smith)

Script: `day168_grid_vs_random_search.py` · every number in this guide is copied by program from `day168_run_output.txt`, the real output of that script.

The textbook says random search beats grid search because it tries more distinct values of the setting that matters. Today tests that claim twice. On a benchmark where we choose how many settings matter, it holds: with one of three settings mattering, random search found the better point in 0.757 of 1,000 problems at 64 trials, and with all three mattering equally only in 0.361. On real gradient boosting on UCI Adult, random search did **not** win: the grid's best was matched or beaten by only 0.325 of 1,000 random searches of 64 trials. A control shows why: learning rate and depth together explain 0.535 of the spread in log loss, far more than either alone, so this problem is not the one-setting-matters case the textbook argument needs.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

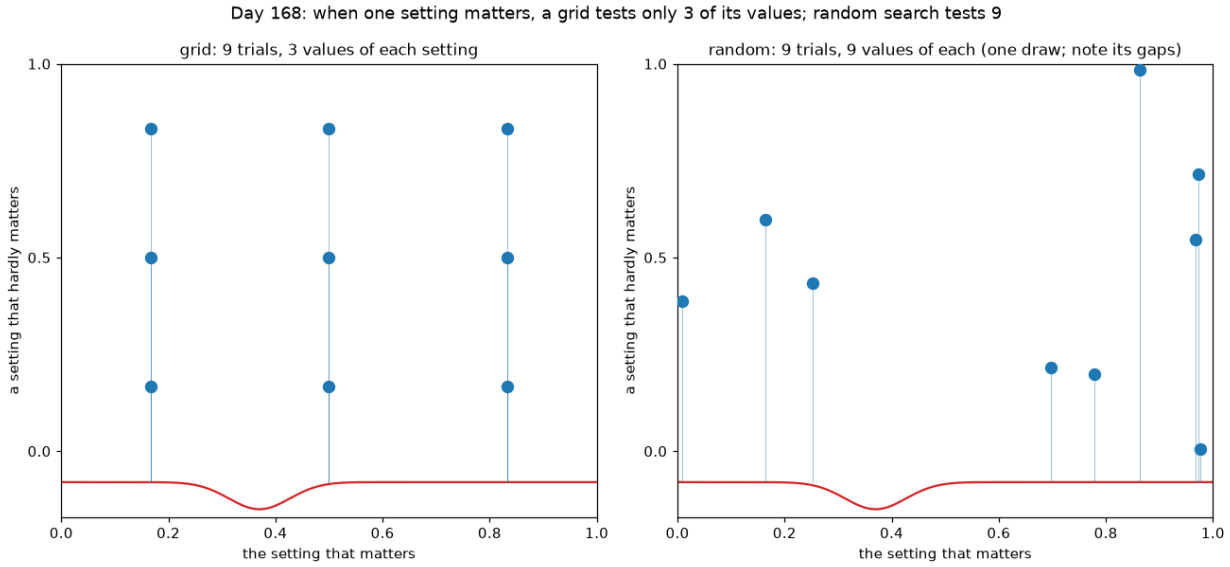
A **hyperparameter** is a setting chosen before training, not learned from the data: a learning rate, a tree depth, the fewest rows a leaf may hold. **Tuning** means trying several settings and keeping the one that scores best on rows the model did not train on.

Grid search picks a few values of each setting and tries every combination: 3 values of 3 settings is 3 × 3 × 3 = 27 trials. **Random search** draws the same number of trials at random from the ranges. Both cost the same per trial; they differ in *where* the trials go.

REAL-WORLD SCENARIO

A baker testing a new bread recipe can bake 27 loaves. A grid tries 3 oven temperatures × 3 baking times × 3 amounts of salt. If only the temperature really matters, 18 of the 27 loaves only repeat temperatures already tried. Random choices would have tried 27 different temperatures.

SYSTEM DIAGRAM



Nine trials in two settings. The grid projects onto only 3 values of the setting that matters; random search projects onto 9, though this draw still misses the dip near 0.37. Drawn by the script from one seeded draw.

Week 25 tunes models, and each day from 168 to 173 adds one tool to a short `dayNNN_exercise.py`; Day 174 uses all six in one reproducible tuning study. The data is Week 24's split of UCI Adult's training file: of the 24,130 fitting rows, 6,000 train each configuration and 6,000 others score it, and the 6,032 validation rows are kept fresh to check the winner. Adult's test file was used once on Day 167 and is not touched. The model is Day 109's gradient boosting with 100 trees, imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. A known-truth benchmark: how many settings matter?

DEFINITION — start here, no prior knowledge assumed

Before real data, a test where the right answer is known. The score of a point p in the unit cube is its weighted squared distance from a hidden best point c, so 0 is perfect. In one problem only the first setting matters (weights 1, 0.01, 0.01); in the other all three matter equally (weights 1, 1, 1).

For each of 1,000 problems, c is drawn at random, the grid and one random search each try n points, and the **shortfall** is the best score each found (the distance from perfect).

REAL-WORLD SCENARIO

Searching a field for a lost ring: if you know it is somewhere along one fence, walk along the fence. A search that spends most of its steps walking across the field wastes them.

MATHEMATICS

$score(p) = \Sigma_j\, w_j\, (p_j - c_j)^2$ $shortfall = min\ over\ the\ trials\ of\ score(p)$

$grid\ of\ k^d\ points:\ only\ k\ distinct\ values\ per\ setting$ $random:\ n\ distinct\ values\ per\ setting$

$P(at\ least\ one\ of\ n\ random\ trials\ lands\ in\ the\ best\ 5\%) = 1 - 0.95^n;\ for\ n = 60\ about\ 0.95$

IMPLEMENTATION

```
for name, w in (("one of three matters", np.array([1.0, 0.01, 0.01])), ("all three matter
    equally", np.ones(3))):
    for n in BUDGETS:
        k = round(n ** (1 / 3)); G = grid_unit(k, 3)
        rg, rr = [], []
        for _ in range(REPEATS):
            c = rng.rand(3)
            f = lambda P: np.sum(w * (P - c) ** 2, 1)
            rg.append(f(G).min()); rr.append(f(random_unit(n, 3, rng)).min())
        rg, rr = np.array(rg), np.array(rr); out[(name, n)] = (rg.mean(), rr.mean())
```

ACTUAL OUTPUT

```
  one of three matters     ,  8 trials: mean shortfall grid 0.02127, random 0.01124; random better in 0.676 of 1000 problems
  one of three matters     , 27 trials: mean shortfall grid 0.00937, random 0.00268; random better in 0.715 of 1000 problems
  one of three matters     , 64 trials: mean shortfall grid 0.00532, random 0.00136; random better in 0.757 of 1000 problems
  all three matter equally ,  8 trials: mean shortfall grid 0.06261, random 0.12378; random better in 0.285 of 1000 problems
  all three matter equally , 27 trials: mean shortfall grid 0.02822, random 0.05002; random better in 0.335 of 1000 problems
  all three matter equally , 64 trials: mean shortfall grid 0.01595, random 0.02587; random better in 0.361 of 1000 problems
Actual output from this lesson's run.
```

What it means

When one setting matters, random search wins most problems at every budget (0.676, 0.715, 0.757) and its average shortfall is about a quarter of the grid's at 64 trials (0.00136 against 0.00532). That is the textbook argument: a grid of 64 tries only 4 values of the setting that matters, random search tries 64.

When all three matter equally the result reverses: the grid wins most problems (0.285, 0.335, 0.361 for random) and its average shortfall is lower (0.01595 against 0.02587 at 64). An evenly spaced grid covers the cube more evenly than random points, which leave gaps and clumps. So the advantage of random search is not a law; it depends on how many settings matter.

WHY CHOOSE THIS?

A benchmark with a known answer can show when a method wins, which real data alone cannot.

WHY NOT THIS? (TRADEOFFS)

Real scores are not quadratic bowls; the benchmark says which property matters, not what real data has.

2. The real thing: grid against random search for boosting on Adult

DEFINITION — start here, no prior knowledge assumed

Three hyperparameters of Day 109's boosting: learning rate from 0.01 to 1 on a log scale, depth from 1 to 8, and the fewest rows a leaf may hold from 1 to 300 on a log scale. Every search works in the unit cube; `to_value` turns a number in [0, 1) into a value on each scale.

Grids of 8, 27 and 64 points (2, 3, 4 values per setting, at the centres of equal slices) are evaluated in full. For random search, a pool of 200 random configurations is evaluated once; each of 1,000 random searches is a random order of that pool, so its best after n trials costs nothing extra to compute.

REAL-WORLD SCENARIO

A company choosing a delivery route can test every combination of three departure times and three roads, or nine random combinations. Which finds the faster route depends on whether the time and the road interact.

SPECIFICATION

Model	Day 109's gradient boosting, log loss, 100 trees, imported unchanged
Rows	6,000 fitting rows train; 6,000 other fitting rows score (log loss); 6,032 validation rows kept fresh
Search space	learning rate 0.01–1 (log), depth 1–8 (whole), min leaf 1–300 (log, whole)
Budgets	8, 27, 64 trials; 1,000 random searches per budget, all drawn from one pool of 200
Control	5 grids of 27 shifted by random amounts within their slices: was the centred grid lucky?

MATHEMATICS

$log: value = lo \cdot (hi / lo)^u$ $int: value = floor(lo + u\,(hi - lo + 1))$

$logint: value = floor(lo \cdot ((hi + 1) / lo)^u)$ $grid\ of\ k: u = (i + 0.5 + shift) / k,\ i = 0 \ldots k - 1$

IMPLEMENTATION

```
def to_value(u, kind, lo, hi):
    if kind == "log":
        return float(lo * (hi / lo) ** u)
    if kind == "int":
        return int(min(hi, np.floor(lo + u * (hi - lo + 1))))
    return int(min(hi, np.floor(np.exp(np.log(lo) + u * (np.log(hi + 1) - np.log(lo))))))
def grid_unit(k, d, shift=None):
    s = np.zeros(d) if shift is None else np.asarray(shift, float)
    axes = [(np.arange(k) + 0.5 + s[j]) / k for j in range(d)]
    return np.array(list(itertools.product(*axes)))
```

IMPLEMENTATION

```
def evaluate(configs, X, y, A, V, F):
    res = []
    with np.errstate(over="ignore"):
        for c in configs:
            t0 = time.time()
            _, E = d109.fit_boost(X[A], y[A], "log", ROUNDS, c["learning rate"], c["depth"],
                evals=(X[V], X[F]), min_leaf=c["min leaf"])
            res.append((d165.log_loss(y[V], d109.sigmoid(E[0][-1])), d165.log_loss(y[F],
                d109.sigmoid(E[1][-1])), time.time() - t0))
    return np.array(res)
```

ACTUAL OUTPUT

```
  6000 rows train each configuration, 6000 score it (log loss), 6032 fresh rows are kept for section 5; 100 trees
  grid of  8: values [[0.0316227766016838, 0.31622776601683794], [3, 7], [4, 72]]; best log loss 0.2973 at {'learning rate':
      0.31622776601683794, 'depth': 3, 'min leaf': 4}
  grid of 27: values [[0.02154434690031884, 0.1, 0.46415888336127803], [2, 5, 7], [2, 17, 116]]; best log loss 0.2934 at
      {'learning rate': 0.1, 'depth': 5, 'min leaf': 2}
  grid of 64: values [[0.01778279410038923, 0.05623413251903491, 0.1778279410038923, 0.5623413251903491], [2, 4, 6, 8], [2,
      8, 35, 147]]; best log loss 0.2924 at {'learning rate': 0.1778279410038923, 'depth': 4, 'min leaf': 2}
  (200 random configurations and 99 grid points evaluated in 1020 s)
  pool's best: log loss 0.2918 at learning rate 0.0956, depth 5, min leaf 1
  pool's best: log loss 0.2924 at learning rate 0.108, depth 4, min leaf 2
  pool's best: log loss 0.2927 at learning rate 0.143, depth 4, min leaf 3
  pool's worst: log loss 0.7371 at learning rate 0.566, depth 8, min leaf 1
```

ACTUAL OUTPUT

```
   8 trials: grid 0.2973; random median 0.2977 (2.5% to 97.5%: 0.2918 to 0.3091); random at least as good in 0.491 of 1000
       searches
  27 trials: grid 0.2934; random median 0.2949 (2.5% to 97.5%: 0.2918 to 0.3007); random at least as good in 0.344 of 1000
      searches
  64 trials: grid 0.2924; random median 0.2924 (2.5% to 97.5%: 0.2918 to 0.2961); random at least as good in 0.325 of 1000
      searches
  5 shifted grids of 27: best 0.2946, 0.2929, 0.2949, 0.2947, 0.2928 (mean 0.2940; the centred grid 0.2934); random of 27 at
      least as good as a shifted grid in 0.410 of comparisons
  the best of all 299 configurations: 0.2918
Actual output from this lesson's run.
```

What it means

The grids found 0.2973, 0.2934 and 0.2924. The median random search found 0.2977, 0.2949 and 0.2924, and matched or beat the grid in 0.491, 0.344 and 0.325 of 1,000 searches. On this problem the grid was as good or better at every budget, the opposite of the textbook claim. The range is wide at 8 trials (2.5% to 97.5%: 0.2918 to 0.3091): with a small budget, random search is a gamble either way.

Was the centred grid lucky? A result that contradicts the textbook needs a control. Five grids of 27 shifted at random within their slices found 0.2946, 0.2929, 0.2949, 0.2947, 0.2928, mean 0.2940 against the centred grid's 0.2934. The centred grid was slightly lucky, and against shifted grids random search matched or beat the grid in 0.410 of comparisons, closer to even, but still not a win for random search.

All differences here are small: the best of all 299 configurations scored 0.2918, and the pool's worst 0.7371 (learning rate 0.566, depth 8, min leaf 1). Choosing a reasonable region matters far more than which search finds the last thousandth.

WHY CHOOSE THIS?

Reusing one evaluated pool for 1,000 random searches makes their spread visible at the cost of 200 fits.

WHY NOT THIS? (TRADEOFFS)

Any search's answer depends on its seed; one random search with a small budget may land anywhere in that spread.

3. Which hyperparameters mattered, and how many values each search tried

DEFINITION — start here, no prior knowledge assumed

The textbook argument needs one setting to matter much more than the others. The pool of 200 random configurations measures this. **Importance** here is eta squared: split one setting's range into 5 equal slices, and ask what share of the scores' total spread is explained by the slice means alone.

A second measure takes learning rate and depth together, in 4 × 4 cells. If the pair explains much more than the two alone add up to, the settings **interact**: the best depth depends on the learning rate.

REAL-WORLD SCENARIO

A car's fuel use depends on speed and on gear, but mostly on the pair: high speed is efficient in top gear and wasteful in first. Studying either alone misses most of the story.

MATHEMATICS

$$\eta^2 = \Sigma_b \, n_b \, (m_b - m)^2 \, / \, \Sigma_i \, (s_i - m)^2$$

n_b, m_b = *configurations and mean score in slice b* $\quad$ m = *mean of all scores* s_i

IMPLEMENTATION

```
def importance(U, scores, bins=5):
    s = np.asarray(scores, float); tot = np.sum((s - s.mean()) ** 2)
    out = []
    for j in range(U.shape[1]):
        b = np.minimum((U[:, j] * bins).astype(int), bins - 1)
        m = np.array([s[b == i].mean() if np.any(b == i) else 0.0 for i in range(bins)])
        n = np.bincount(b, minlength=bins)
        out.append(float(np.sum(n * (m - s.mean()) ** 2) / tot))
    return np.array(out)
```

ACTUAL OUTPUT

```
  learning rate: explains 0.293 of the spread in log loss on its own
  depth        : explains 0.046 of the spread in log loss on its own
  min leaf     : explains 0.056 of the spread in log loss on its own
  learning rate and depth together (4 x 4 cells): explain 0.535
  in the better half of the pool, learning rate ran from 0.023 to 0.785; outside it from 0.010 to 0.985
   8 trials: grid tries 2 values of each hyperparameter; random tries 8 learning rate, 5 depth, 6 min leaf (first random
       order)
  27 trials: grid tries 3 values of each hyperparameter; random tries 27 learning rate, 8 depth, 20 min leaf (first random
      order)
  64 trials: grid tries 4 values of each hyperparameter; random tries 64 learning rate, 8 depth, 38 min leaf (first random
      order)
Actual output from this lesson's run.
```

What it means

Learning rate explains 0.293 on its own, depth 0.046 and min leaf 0.056. Learning rate matters most, but not alone: learning rate and depth together explain 0.535, against 0.293 + 0.046 = 0.339 for the two separately. Finer cells explain a little more by chance alone (with 16 cells and 200 points, about 15/199 = 0.08 when nothing matters), which does not account for a gap of 0.2. The scatter in the figure shows the interaction: deep trees do well at low learning rates and badly at high ones, shallow trees the reverse.

A grid of 64 tries 4 values of each setting; the first random search tried 64 learning rate, 8 depth and 38 min leaf values. Random search did try more learning rates, but with an interaction the grid's even coverage of the learning rate and depth pair may count as much. A likely reason the grid held up, consistent with the benchmark but not tested directly here, is that this problem's effective dimension is closer to two than one.

Importance from 200 points is itself noisy, and the good region is wide: in the better half of the pool, learning rate ran from 0.023 to 0.785.

WHY CHOOSE THIS?

Measuring importance after a random search tells you which settings to tune more finely next time.

WHY NOT THIS? (TRADEOFFS)

One-at-a-time importance misses interactions; check pairs too, and treat values from a few hundred trials as rough.

4. What each search costs

DEFINITION — start here, no prior knowledge assumed

Every trial is one model fit, but trials are not equal: deeper trees take longer. The pool's times are grouped by depth, and each search's total time is added up.

REAL-WORLD SCENARIO

Two plumbers who each visit 27 houses do not work the same hours if one is sent to the big houses.

ACTUAL OUTPUT

```
  depth 1: 0.66 s per configuration on average (23 in the pool)
  depth 4: 2.53 s per configuration on average (21 in the pool)
  depth 8: 6.76 s per configuration on average (32 in the pool)
   8 trials: grid 26 s; random 23 s (first random order)
  27 trials: grid 78 s; random 105 s (first random order)
  64 trials: grid 200 s; random 238 s (first random order)
  pool of 200: 717 s in all
Actual output from this lesson's run.
```

What it means

A depth-8 fit took 6.76 s on average against 0.66 s at depth 1, about ten times as long. In this run the grid of 64 took 200 s and the first random search of 64 took 238 s; a likely reason, not tested here, is that this search drew more deep trees.

A first-draft finding about timing. The first run of this same script gave identical scores but times about 2.4 times smaller (0.28 s at depth 1, 2.74 s at depth 8), and in it the grid of 64 took longer than the random search (170 s against 101 s). Wall-clock times on this machine swing between runs, so only comparisons within one run are reported. What held in both runs is the ratio: depth 8 costs about ten times depth 1, so a search's cost depends on how often it visits expensive settings.

WHY CHOOSE THIS?

Counting trials is a fair budget when every trial costs about the same.

WHY NOT THIS? (TRADEOFFS)

When cost depends on the settings, a budget in seconds is fairer, and a search can avoid expensive regions.

5. The winner's curse: the best score on its own rows against fresh rows

DEFINITION — start here, no prior knowledge assumed

The configuration that scores best on the rows used to choose it is partly the best and partly the luckiest. Its score on those rows is therefore optimistic. Day 164 showed this for picking the best of 100 models; here each configuration is also scored on the 6,032 fresh validation rows that no search looked at.

REAL-WORLD SCENARIO

The fund manager with the best record last year is usually good and also lucky; next year's return tends to be lower.

ACTUAL OUTPUT

```
  over all 200 configurations: tuning and fresh log loss correlate 0.996; mean fresh - tuning +0.0056
   8 trials: random winner's fresh - tuning +0.0054 on average, -0.0002 beyond the average configuration's (fresh log loss
       0.3040); grid winner +0.0034 (fresh 0.3007)
  27 trials: random winner's fresh - tuning +0.0064 on average, +0.0008 beyond the average configuration's (fresh log loss
      0.3011); grid winner +0.0086 (fresh 0.3020)
  64 trials: random winner's fresh - tuning +0.0074 on average, +0.0018 beyond the average configuration's (fresh log loss
      0.3005); grid winner +0.0069 (fresh 0.2993)
  the pool's winner on tuning rows ranks 9 of 200 on fresh rows (fresh 0.3010; best on fresh 0.2971)
Actual output from this lesson's run.
```

What it means

Tuning and fresh scores correlate 0.996, and every configuration does slightly worse on fresh rows (by +0.0056 on average). Beyond that, the random winner's extra optimism grows with the budget but stays tiny: -0.0002, +0.0008, +0.0018 at 8, 27 and 64 trials. More trials means more chances to be lucky.

The pool's overall winner on tuning rows ranks 9 of 200 on fresh rows (0.3010 against the best fresh 0.2971). With 6,000 scoring rows and well-behaved models the curse is small here; with fewer scoring rows or thousands of trials it grows. Day 172 returns to this with cross-validation.

WHY CHOOSE THIS?

Scoring the winner once on rows no search touched gives an honest estimate of its performance.

WHY NOT THIS? (TRADEOFFS)

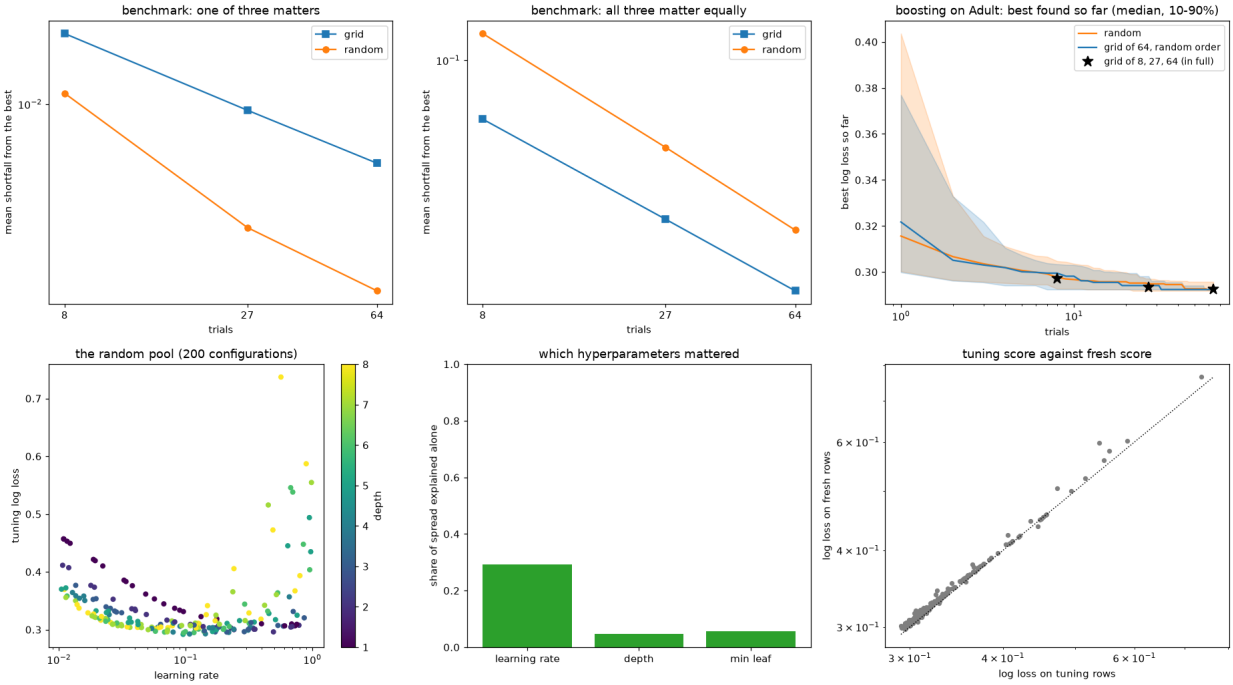
Those rows cannot then be used to choose anything; a second round of tuning needs new fresh rows.

Today's part for the week's build

`day168_exercise.py` holds the search space and six functions: `to_value`, `grid_unit`, `random_unit`, `decode`, `best_so_far` and `importance`, copied unchanged. Its hand checks, each worked on paper first:

ACTUAL OUTPUT

```
to_value log 0.01 to 1: u = 0 -> 0.0100 (by hand 0.01), u = 0.5 -> 0.1000 (0.01 x 100^0.5 = 0.1)
to_value int 1 to 8: u = 0 -> 1, u = 0.5 -> 5 (floor(1 + 0.5 x 8) = 5), u = 0.999 -> 8 (8)
to_value logint 1 to 300: u = 0.5 -> 17 (floor(e^(0.5 ln 301)) = floor(17.35) = 17)
grid_unit(2, 2): [[0.25, 0.25], [0.25, 0.75], [0.75, 0.25], [0.75, 0.75]] (by hand the centres
    0.25 and 0.75 in every combination)
grid_unit(2, 2, shift [0.4, 0]): [[0.45, 0.25], [0.45, 0.75], [0.95, 0.25], [0.95, 0.75]] (first
    axis (0.5 + 0.4)/2 = 0.45 and (1.5 + 0.4)/2 = 0.95)
random_unit(2, 3, seeded): [[0.5488, 0.7152, 0.6028], [0.5449, 0.4237, 0.6459]] (2 rows of 3, all
    in [0, 1))
decode([[0.5, 0.5, 0.5]]): [{'learning rate': 0.1, 'depth': 5, 'min leaf': 17}] (by hand 0.1, 5,
    17)
best_so_far([3, 1, 2, 0.5]): [3.0, 1.0, 1.0, 0.5] (by hand 3, 1, 1, 0.5)
importance with 2 bins, score = 1 exactly when the first setting is high: [1.0, 0.0] (by hand 1
    and 0)
```



Top: the benchmark's mean shortfall when one setting matters and when all three do; the best log loss found so far on Adult (median and 10–90% of 1,000 searches). Bottom: the random pool's learning rate against log loss, coloured by depth; importance; every configuration's tuning score against its fresh score.

Interview preparation — technical questions

Q: How would you compare two tuning methods fairly?

A: Same budget, same scoring rows, many repeats (here 1,000 random orders of one evaluated pool), and a known-truth benchmark alongside the real problem.

Q: Random search did not beat the grid here. Why?

A: Learning rate and depth interact (together 0.535, separately 0.339), so this is not the one-setting-matters case; a shifted-grid control showed the centred grid was only slightly lucky.

Q: How do you put a learning rate into a search?

A: On a log scale: draw u uniformly and take lo · (hi/lo)^{u}, so 0.01 to 0.1 gets as many trials as 0.1 to 1.

Q: Why is the tuning score of the winner optimistic?

A: It was chosen for scoring best on those rows, partly by luck; score it once on fresh rows.

Q: What does a budget of n trials hide?

A: Trials can cost very different amounts: depth 8 cost about ten times depth 1 here.

Interview preparation — theoretical questions

Q: Why does random search usually beat grid search?

A: When only a few settings matter, a grid wastes trials repeating the same values of them; random search tries a new value every time.

Q: When is a grid better?

A: When all settings matter about equally and the budget is small, its even coverage leaves fewer gaps than random points.

Q: What is effective dimension?

A: The number of settings that really change the score; random search's advantage grows as it falls below the number searched.

Q: Is there something between the two?

A: Low-discrepancy sequences (Sobol, Halton) spread points evenly and give a new value of every setting each trial; Day 169's Bayesian optimisation uses earlier trials to choose the next.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Random search beats grid search when few settings matter, and the known-truth benchmark shows it clearly. On real boosting on Adult, two settings interacted and the grid held its own. Measure which settings matter, put scales like the learning rate on a log axis, compare searches over many repeats, and check the winner on fresh rows.

- Benchmark, one setting matters: random better in 0.757 of problems at 64 trials; all matter: 0.361.
- Adult: random matched or beat the grid in 0.325 of searches at 64 trials; shifted-grid control 0.410.
- Learning rate alone 0.293; with depth 0.535: an interaction.
- Winner's extra optimism on fresh rows: +0.0018 at 64 trials; the pool's winner ranks 9 of 200.

Suggested watch order, if starting from zero

- 1. Hyperparameters Tuning: Grid Search vs Random Search (DataMListic) — a short comparison of the two searches.
- 2. IML17: Grid vs random search for hyperparameter tuning? (Bevan Smith) — a lecture on the same question.

Day 169 preview

- **Bayesian optimisation from scratch.** A Gaussian process learns the score from the trials so far and chooses the next one, compared with today's random search on the same budget.